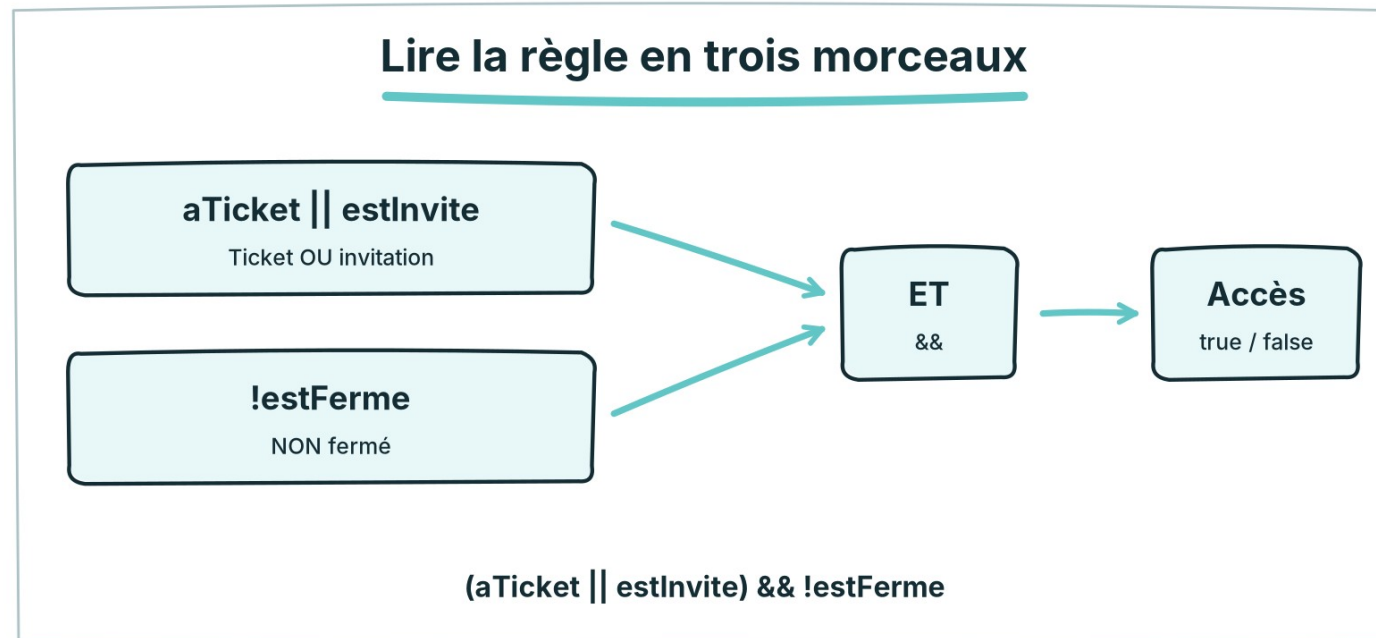


Deux règles. Une décision.

S04 · Logique booléenne et conditions combinées



20 minutes. Quatre programmes.

3 min : lancer · 4 min : transmettre · 5 min : renvoyer · 6 min : réutiliser · 2 min : remettre

3 min

Lancer

`annoncerDepart()`

4 min

Transmettre

`saluer(prenom)`

5 min

Renvoyer

`doubler(nombre)`

6 min

Réutiliser

`additionner(a, b)`

Puis 2 minutes : conserver le travail, remettre, vérifier le reçu.

Écris → exécute → compare.

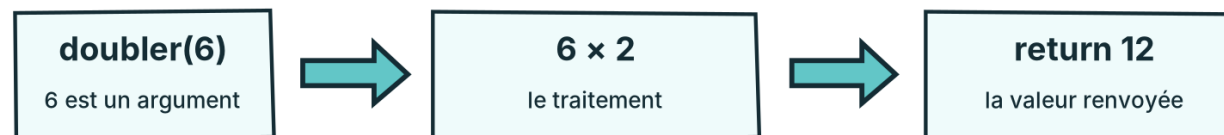
Un test raté donne une piste. Corrige, puis teste à nouveau.

Une fonction : une entrée, un résultat.

DÉCLARER

```
function doubler(nombre) { return nombre * 2; }
```

↑ paramètre : le nom de l'entrée



```
const resultat = doubler(6);
```

La variable garde 12 : le programme peut la réutiliser.

console.log(resultat) affiche 12. Ce n'est pas return.

Afficher ≠ renvoyer.

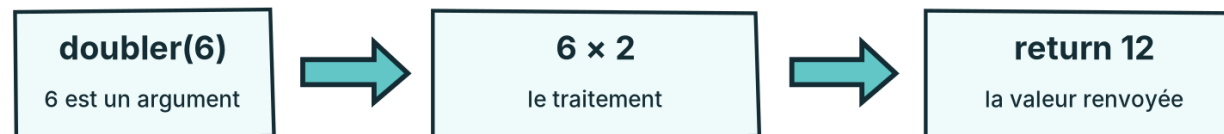
Une sortie visible. Une valeur à réutiliser.

Une fonction : une entrée, un résultat.

DÉCLARER

```
function doubler(nombre) { return nombre * 2; }
```

↑ paramètre : le nom de l'entrée



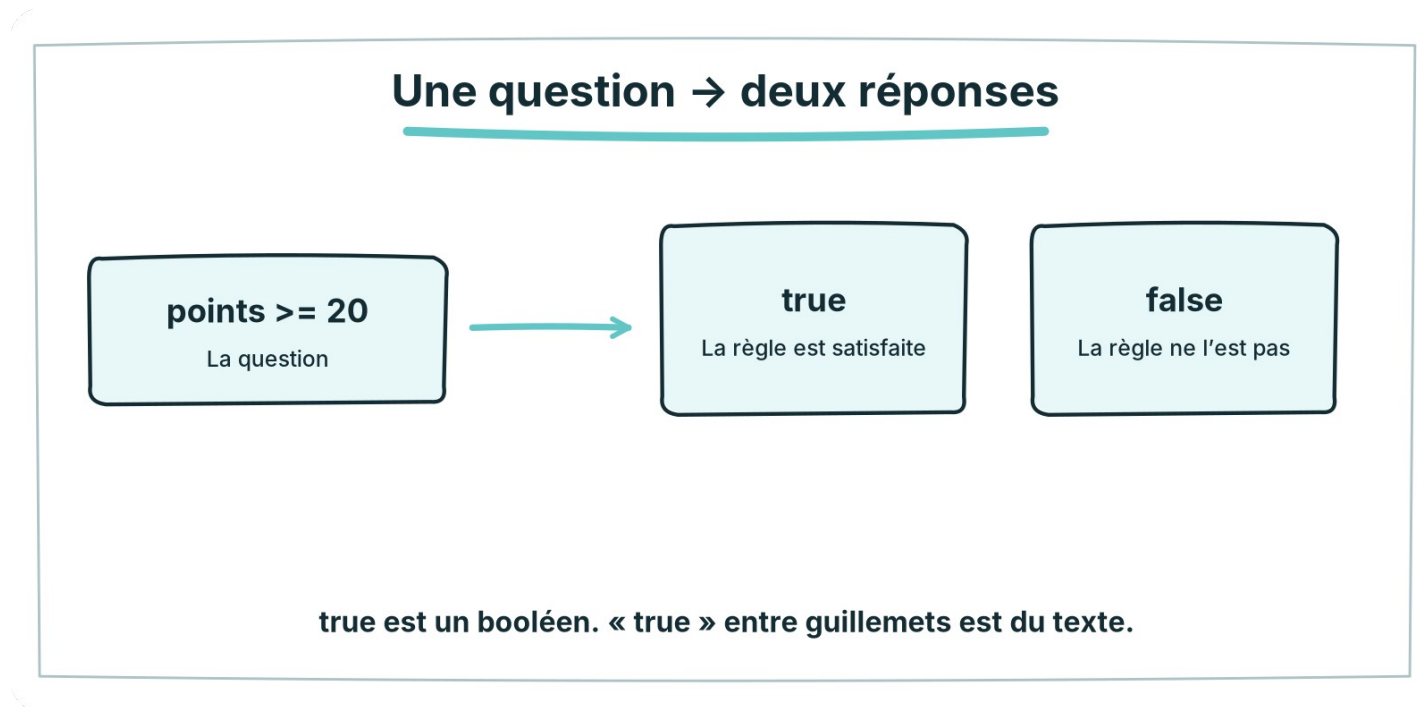
```
const resultat = doubler(6);
```

La variable garde 12 : le programme peut la réutiliser.

console.log(resultat) affiche 12. Ce n'est pas return.

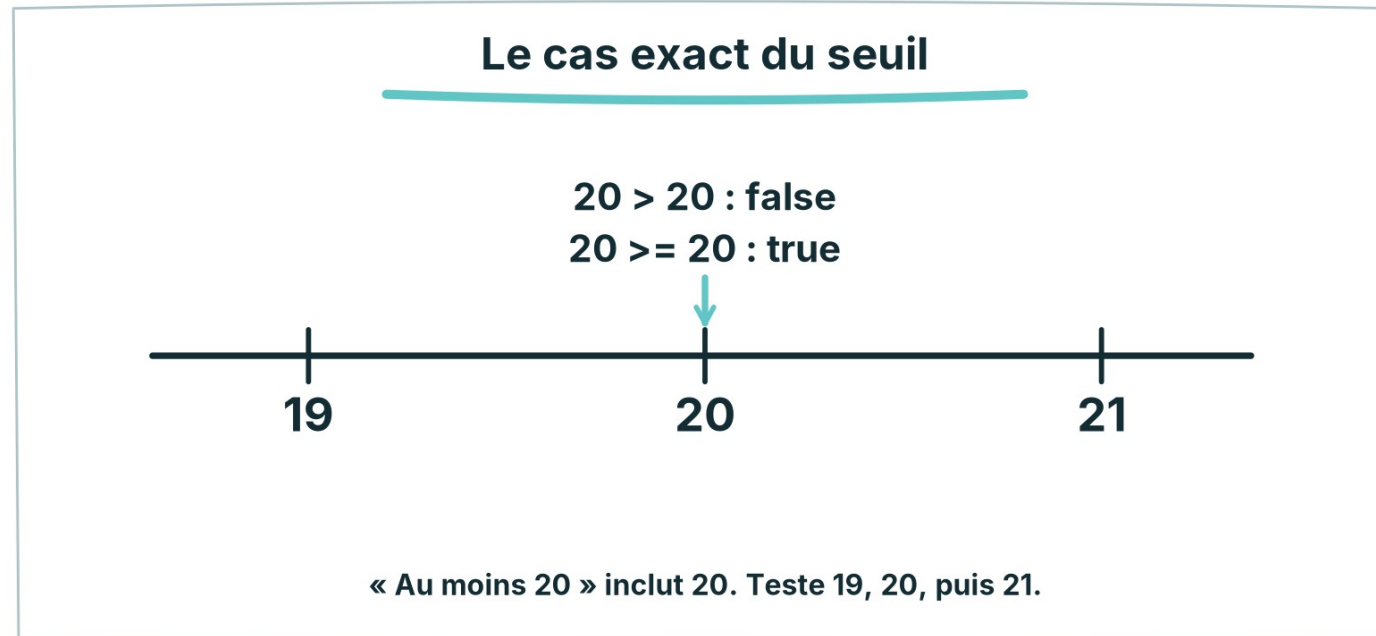
Un booléen : deux valeurs.

true ou false. Pas « peut-être ».



Le piège du seuil.

$20 > 20$? $20 \geq 20$?



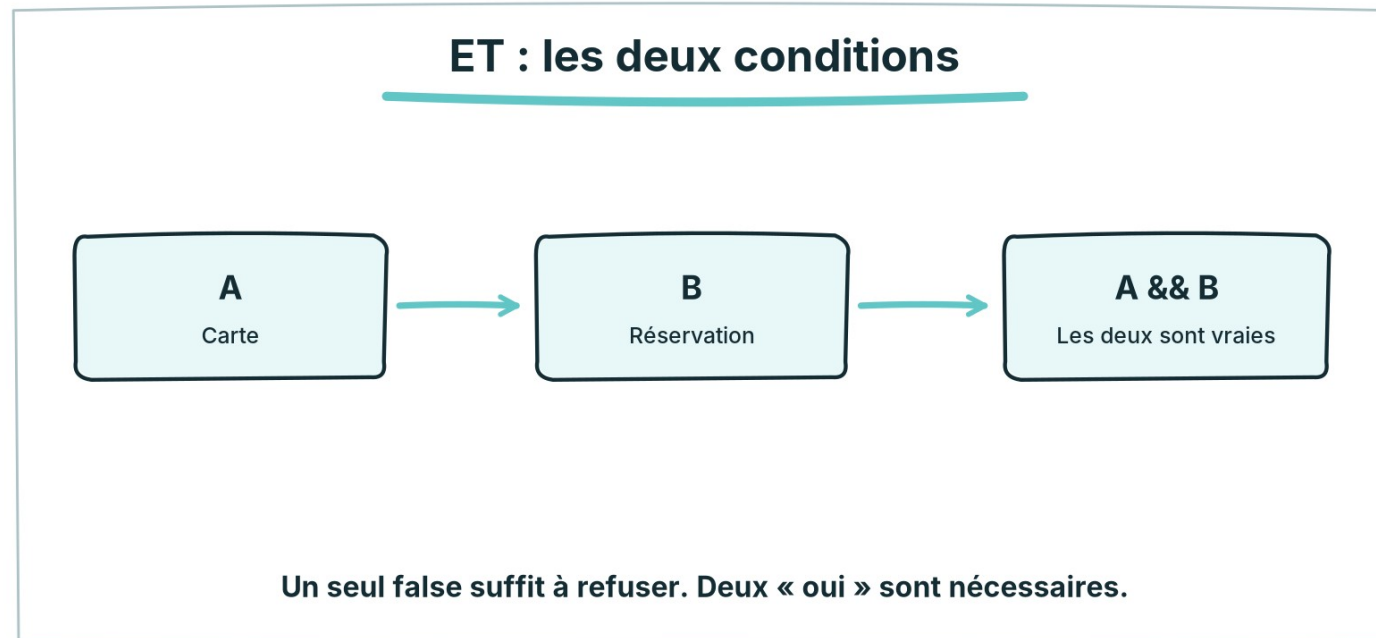
À toi : comparer.

Testez juste avant, au seuil, juste après.

À toi de jouer.

ET : les deux.

Carte ET réservation.



Ne teste pas seulement « oui ».

Quatre combinaisons à vérifier.

Deux booléens : quatre combinaisons

A : false

B : false

A : false

B : true

A : true

B : false

A : true

B : true

Un test réussi ne suffit pas. Couvre aussi les cas qui refusent.

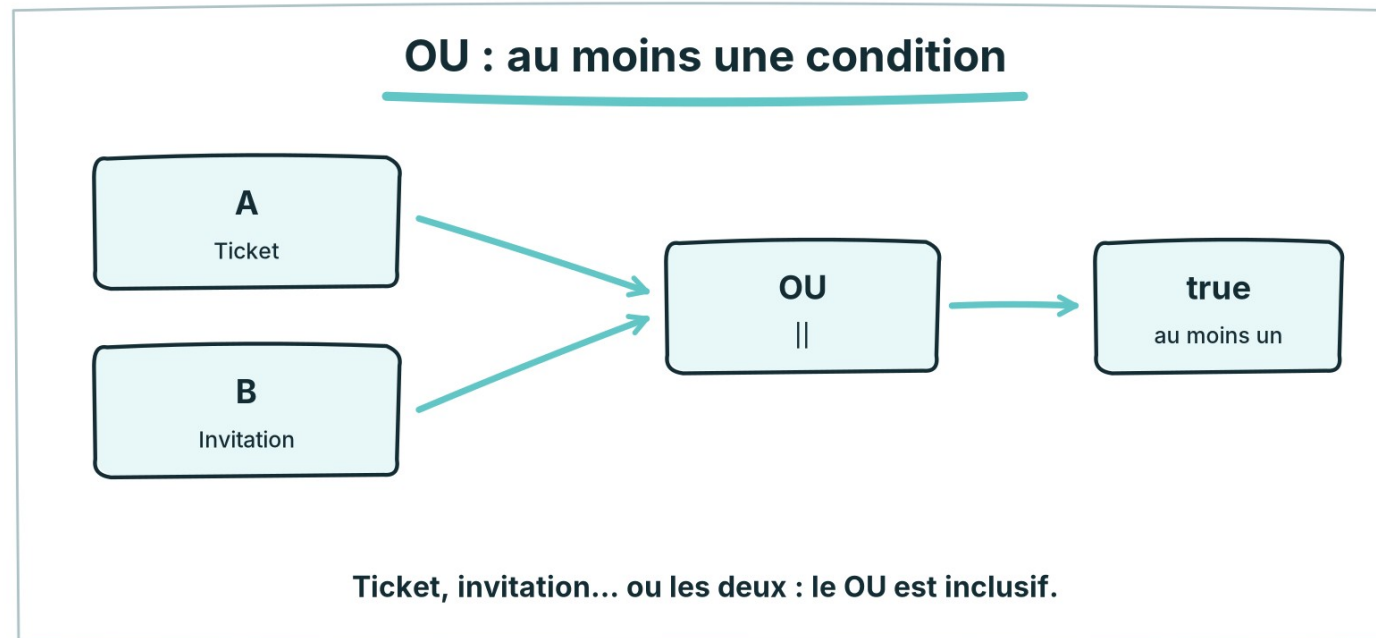
À toi : ouvrir la salle.

Prévois → manipule → vérifie.

À toi de jouer.

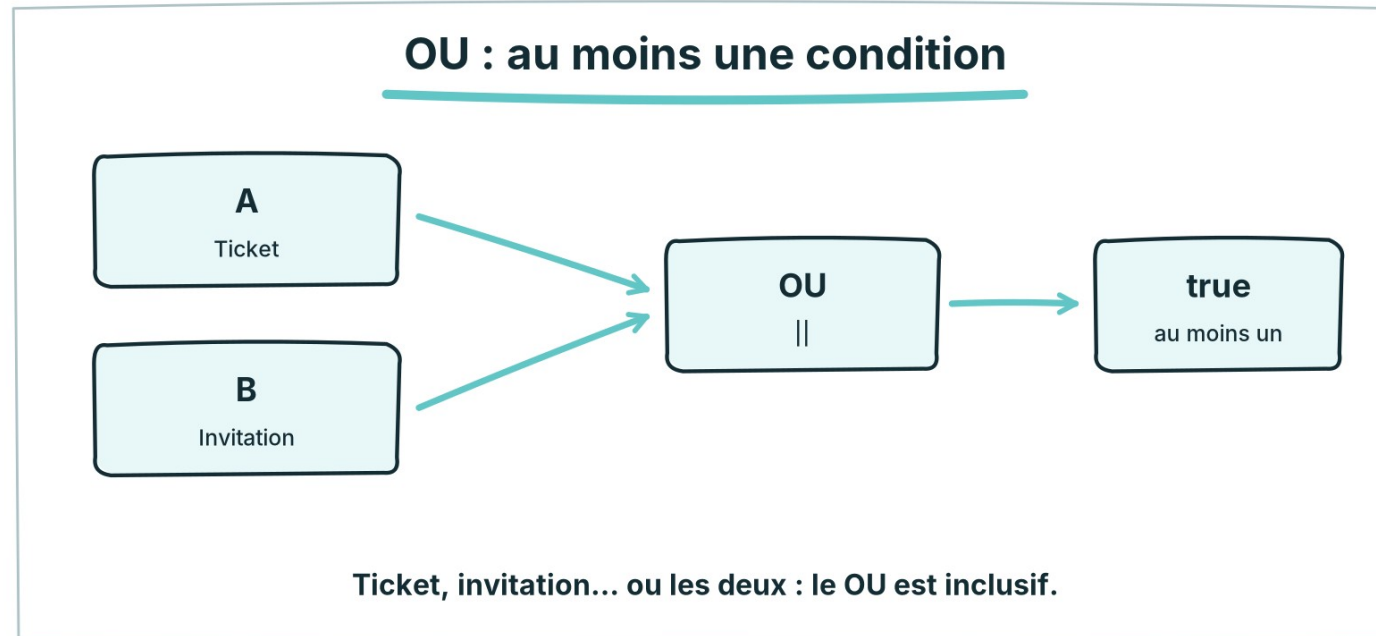
OU : au moins un.

Ticket, invitation... ou les deux.



Et si les deux sont vrais ?

Le OU accepte encore.



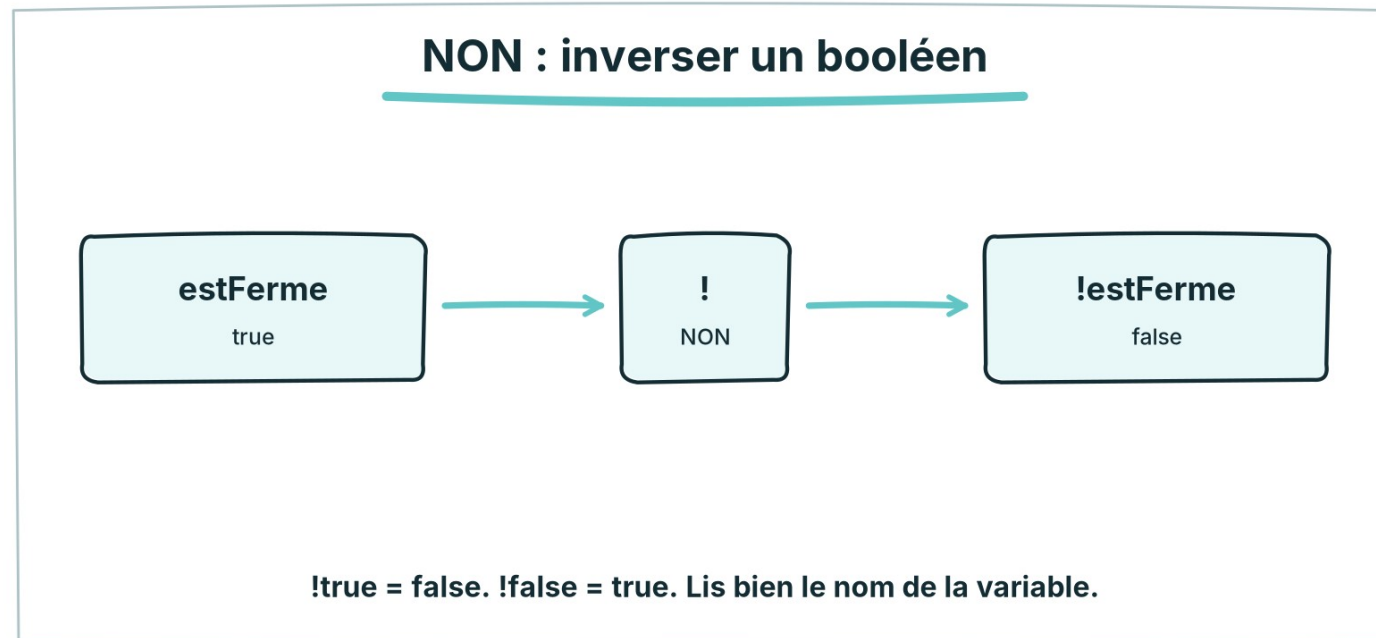
Pause · 15 min

Reprends à l'annonce du professeur.

15 minutes de pause

NON : inverser.

`!estFerme` se lit « pas fermé ».



Les parenthèses délimitent.

!(A && B) n'est pas !A && B.

Qu'est-ce que tu inverses ?

!(A && B)

NON porte sur le groupe entier

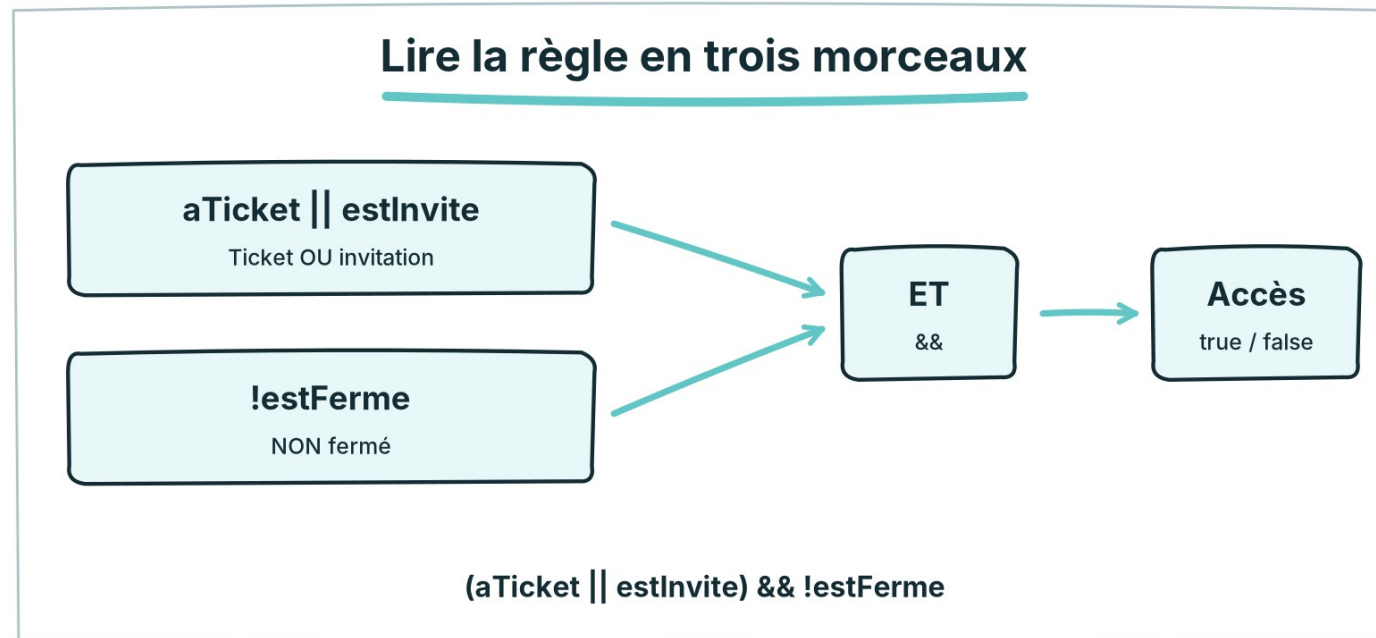
!A && B

NON porte seulement sur A

Les parenthèses montrent le groupe sur lequel porte la règle.

On combine les règles.

(ticket OU invitation) ET NON fermé.



Huit cas. Pas de devinette.

Trois booléens → huit combinaisons.

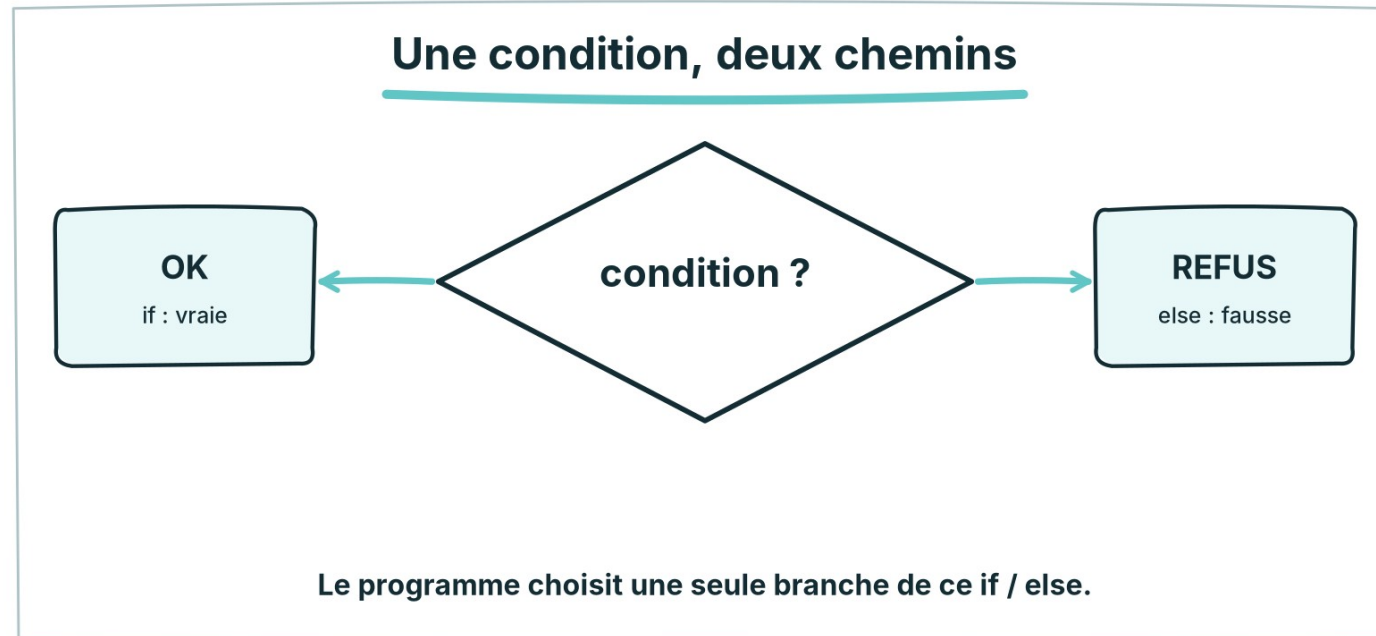
Trois booléens : huit combinaisons

aTicket	estInvite	estFerme
false	false	false
false	false	true
false	true	false
false	true	true
true	false	false
true	false	true
true	true	false
true	true	true

Trois entrées : $2 \times 2 \times 2 = 8$ cas à vérifier.

if / else choisit un chemin.

Une condition. Deux messages.



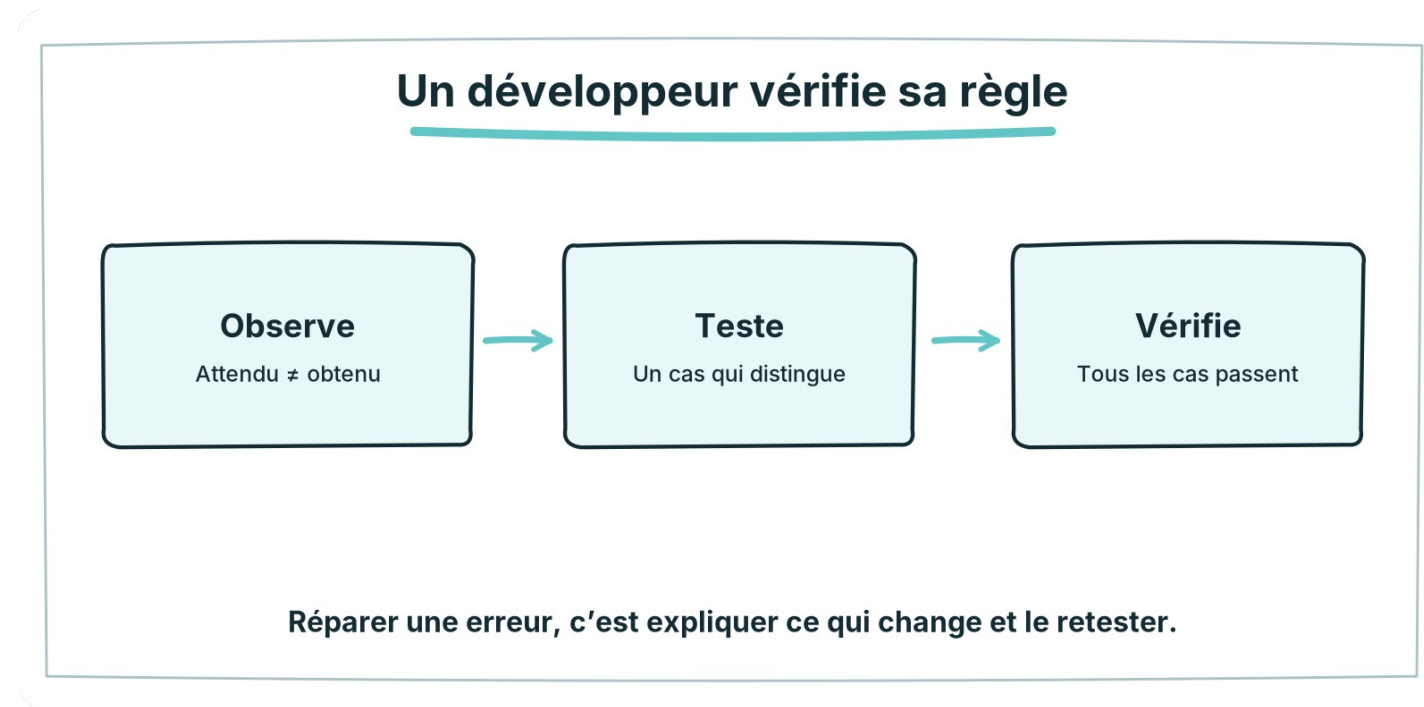
À toi : passer au code.

Le cadre est fourni. Écris la condition.

À toi de jouer.

Trouve le cas qui casse.

Le programme accepte une carte seule.



Répare, puis reteste.

Un changement. Tous les cas.

À toi de jouer.

Trois niveaux. Une méthode.

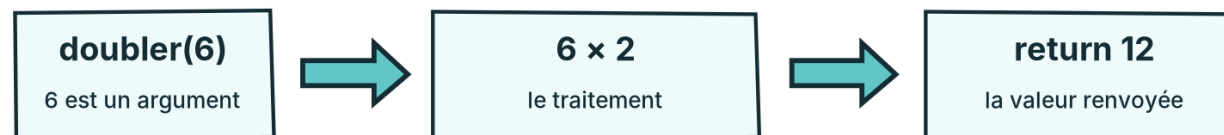
Seuil → ET → règle complète.

Une fonction : une entrée, un résultat.

DÉCLARER

```
function doubler(nombre) { return nombre * 2; }
```

↑ paramètre : le nom de l'entrée



```
const resultat = doubler(6);
```

La variable garde 12 : le programme peut la réutiliser.

console.log(resultat) affiche 12. Ce n'est pas return.

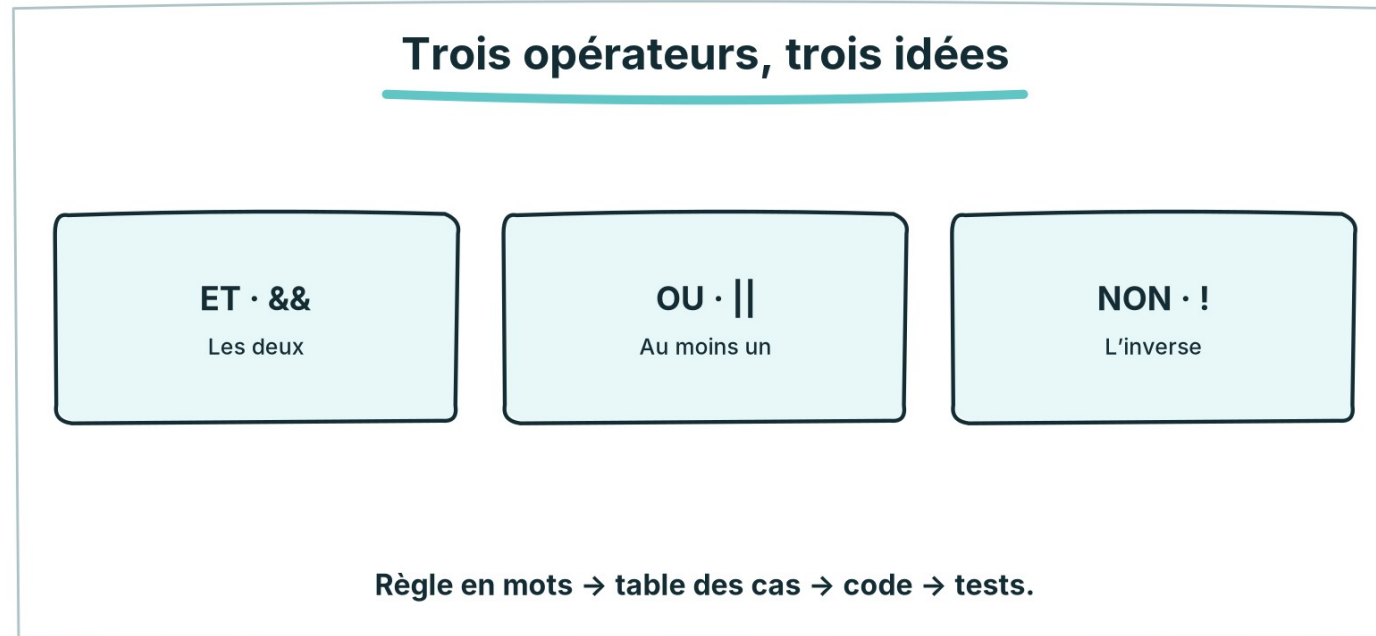
Point de maîtrise · 15 min

Seul. Sans aide. Sans mémo.

À toi de jouer.

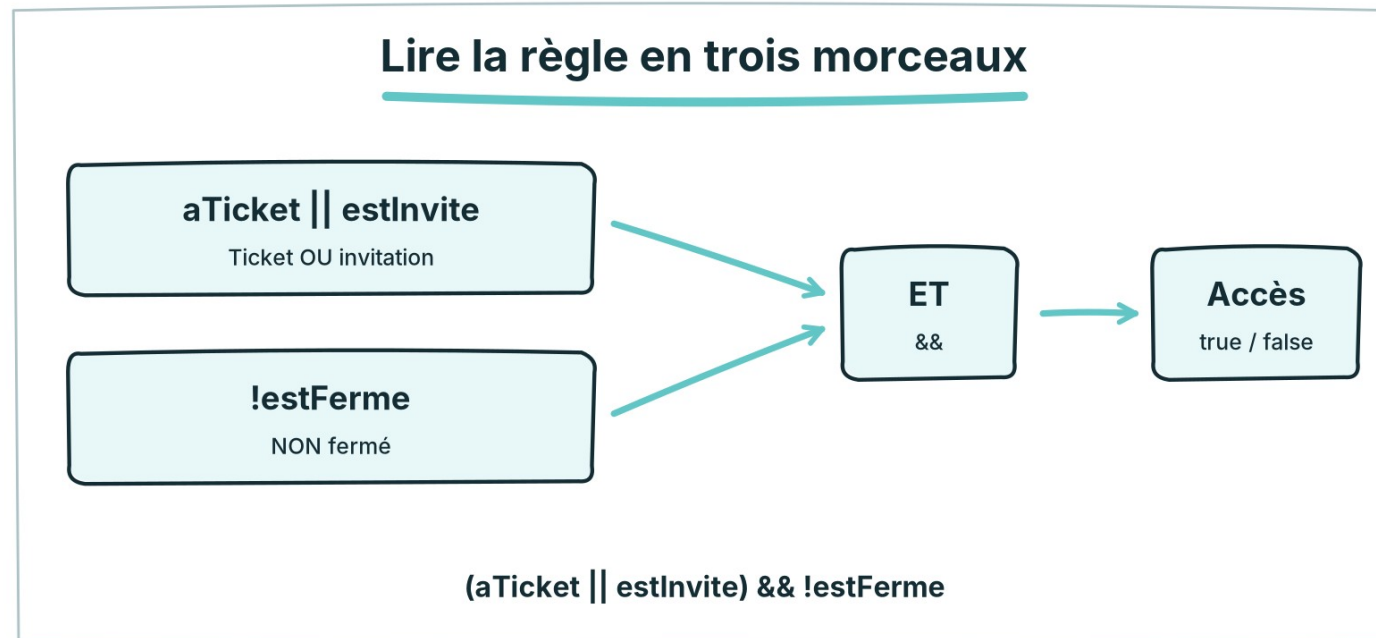
Trois opérateurs, une méthode.

ET = les deux · OU = au moins un · NON = inverse



Pour ceux qui ont terminé.

Le gardien des règles.



Et pour aller plus loin ?

Un ticket, au moins 14 ans ou un accompagnateur... et un lieu ouvert.

